COMMUNITY HERO: HYPERLOCAL CIVIC PROBLEM SOLVER & SMART CITY INCIDENT RESOLUTION PLATFORM WITH 3D GEOSPATIAL VISUALIZATION AND MACHINE LEARNING

PROJECT REPORT [Phase-III]

Submitted in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

Department of Computer Science and Engineering

Submitted to

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY BHOPAL (M.P.)

Submitted by

Aarush Bharti (24U020010 and 5th Semester)
Ashish Goutam (24U020011 and 5th Semester)

Under the supervision of

Dr. Sreemoyee Biswas
Assistant Professor
(CSE)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)

CERTIFICATE

This is to certify that the investigation and engineering work documented in this report entitled "Community Hero: Hyperlocal Civic Problem Solver & Smart City Incident Resolution Platform with 3D Geospatial Visualization and Machine Learning" has been carried out by AARUSH BHARTI (24U020010) and ASHISH GOUTAM (24U020011). It constitutes an authentic piece of academic engineering, conducted under our supervision in the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal for the partial fulfillment of the Bachelor of Technology during the academic year 2026-27.

The web application, trained classification model, 75-meter spatial duplicate detection algorithm, voice intake system, and testing phases have been evaluated as part of the Phase III minor project deliverables.

Date:

Dr. Sreemoyee Biswas
Assistant Professor
(CSE)
IIIT Bhopal (M.P.)

Dr. Rahul Chaurasia
Department of CSE,
IIIT Bhopal (M.P.)

INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
BHOPAL (M.P.)

DECLARATION

We hereby declare that the following project report entitled “Community Hero: Hyperlocal Civic Problem Solver & Smart City Incident Resolution Platform with 3D Geospatial Visualization and Machine Learning” presented in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is an authentic documentation of our original work, carried out under the guidance of Dr. Sreemoyee Biswas. The work has been carried out at the Indian Institute of Information Technology, Bhopal. The project work presented has not been submitted in part or whole for the award of any other degree or diploma in any other institute.

The software codebase, 3D globe display, machine learning model, voice helpline module, database schema, and test results presented in this report were developed and evaluated by us. All external reference materials and open-source libraries have been formally cited according to IEEE standards.

We declare that the facts mentioned above are true to the best of our knowledge. In case of any discrepancy that may occur, we will be the ones to take responsibility.

Aarush Bharti (24U020010)
Ashish Goutam (24U020011)
5th Semester, Department of CSE
Indian Institute of Information Technology, Bhopal

ACKNOWLEDGEMENT

We, Aarush Bharti and Ashish Goutam, express our profound gratitude to all the mentors, faculty members, and peers who guided and supported our efforts during the development of our minor project titled “Community Hero: Hyperlocal Civic Problem Solver & Smart City Incident Resolution Platform with 3D Geospatial Visualization and Machine Learning”.

First and foremost, we extend our heartfelt thanks to our Director, Dr. Ashutosh Kumar Singh, for providing the laboratory facilities and academic environment needed to carry out this project.

We are deeply grateful to our Project Guide, Dr. Sreemoyee Biswas, Assistant Professor, Department of Computer Science and Engineering, for her constant guidance, valuable suggestions, and encouragement. Her support helped us improve our technical concepts, software design, and practical testing.

Our sincere thanks to the Department Coordinator, Dr. Rahul Chaurasia, for the support provided during this semester. We also thank all faculty members of the CSE department for their helpful advice.

A special thanks to our batchmates and friends who helped test the web interface and voice features and provided useful feedback. Finally, we thank our parents and family members for their constant patience and moral support throughout our studies.

Aarush Bharti (24U020010)
Ashish Goutam (24U020011)

TABLE OF CONTENTS

Certificate ii
Declaration iii
Acknowledgement iv
List of Figures vi
List of Tables vii
1. ABSTRACT 8
2. INTRODUCTION 9
3. LITERATURE REVIEW 11
4. PROBLEM DEFINITION AND OBJECTIVES 13
5. PROPOSED METHODOLOGY AND ARCHITECTURE 14
6. PROPOSED ALGORITHMS AND PSEUDOCODE 16
7. SYSTEM DIAGRAMS AND WORKFLOWS 17
8. IMPLEMENTATION AND TECHNOLOGIES 21
9. RESULTS AND TESTING 23
10. CONCLUSION AND FUTURE SCOPE 25
11. REFERENCES 26
12. LIST OF PUBLICATION 27
13. PLAGIARISM REPORT (TURNITIN) 28

LIST OF FIGURES

Fig. 1. High-Level Full-Stack System Architecture of Community Hero 17
Fig. 2. Data Flow Diagram (DFD Level-0 & Level-1) 18
Fig. 3. Interactive 3D Virtual Earth and Incident Visualization 18
Fig. 4. Multilingual Voice Intake & Automated Ticket Dispatch Pipeline 19
Fig. 5. Municipal Command Center Real-Time Dashboard 19
Fig. 6. Citizen Hyperlocal Geospatial Incident Map and Community Cards 20
Fig. 7. Field Repair Work Order Tracking and Status Verification 20

LIST OF TABLES

Table 1. Summary of reviewed works and their relevance to this project 12
Table 2. System requirements and technical specifications 13
Table 3. Primary database tables and integrity rules 15
Table 4. Technology stack and roles in Community Hero 21
Table 5. Key REST API endpoints implemented in backend 22
Table 6. Offline NLP classification performance across five categories 23
Table 7. Spatial duplicate detection accuracy across distance tests 23
Table 8. Functional test cases and validation results (TC1 to TC6) 24
Table 9. Server response latency benchmarks under concurrent load 24

1. ABSTRACT

Modern urban governance faces chronic operational bottlenecks when logging and remediating neighborhood infrastructure defects such as damaged road pavements, non-functioning street luminaires, potable water line fractures, uncollected municipal waste, and dangerous overhead wiring. Prevailing municipal grievance mechanisms in developing regions rely on antiquated, static web forms that leave citizens without transparency, while inundating administrative desks with hundreds of redundant tickets for identical physical occurrences. To bridge this divide, this Phase III engineering project introduces Community Hero, an original full-stack civic response architecture that transforms citizen incident logging into an intuitive, visually verified, and automated operational workflow.

The client architecture is engineered using React 18, utilizing a Three.js hardware-accelerated 3D digital twin of the Earth for regional incident telemetry, seamlessly coordinated with a Leaflet 2D street map for meter-accurate GPS pin placement, photographic evidence uploading, and citizen upvoting. The server tier is implemented with Node.js and Express, coupled with an embedded SQLite relational database operating in Write-Ahead Logging mode for fast, atomic data transactions.

To bypass recurring cloud API subscription costs and eliminate external network latency, an offline supervised natural language processing model was trained locally via Scikit-Learn using bi-gram TF-IDF vectorization and a Multinomial Naive Bayes classifier. Evaluated against 1,200 labeled municipal grievance descriptions across five primary civic failure classes, the embedded engine achieves an empirical classification accuracy of 89.2% with a sub-5 millisecond inference runtime. Redundant ticket dispatch is eliminated by a 75-meter spatial clustering algorithm based on the Haversine spherical geodesic formula, grouping contiguous incident reports under a unified parent work order while aggregating community priority weight.

To address accessibility for illiterate or non-English speaking citizens, an automated voice intake helpline accepts spoken audio in Hindi and English, extracting geographic landmarks and incident context to generate structured Open311-compliant work orders. Real-time telemetry is broadcast to municipal staff consoles via Server-Sent Events (SSE). Extensive load benchmarks verify response latencies below 25 milliseconds, validating Community Hero as an efficient, cost-effective digital twin solution for smart cities.

Keywords: Civic-Tech, Three.js Digital Twin, React 18, Node.js, SQLite WAL, Offline NLP Triage, Haversine Spatial Deduplication, Multilingual Voice Telephony, Open311.

2. INTRODUCTION

2.1 Background and Real-World Context

Across rapidly urbanizing Indian municipalities, physical infrastructure decay such as road craters, malfunctioning streetlights, fractured water pipelines, and overflowing waste bins presents severe public hazards. Conventional grievance redressal channels rely on manual helplines, physical visits, or unstructured web forms that offer zero status transparency. Concurrently, municipal authorities receive hundreds of unorganized, redundant complaints during severe weather for identical incidents, resulting in duplicate dispatches and squandered municipal budgets.

2.2 Need for Voice Accessibility

A critical barrier in contemporary e-governance applications is the digital literacy gap. Many elderly, underprivileged, or non-technical residents find smartphone menu navigation and typing English descriptions challenging. Community Hero overcomes this barrier through an integrated voice telephone and browser intake module that accepts natural spoken audio in Hindi or English, extracts landmark names and problem descriptions, and generates structured tickets without manual operator overhead.

2.3 Project Motivation and Objectives

Engineering Objectives: As undergraduate computer science engineers, our objective was to design and deploy an open-source, resilient, and responsive municipal incident redressal platform. The primary technical objectives comprise:

- **1. Spatial Interface:** Dual Geospatial Interface: Construct a WebGL 3D planetary digital twin paired with a 2D street map for intuitive spatial incident tracking.
- **2. Embedded Classifier:** Offline NLP Intelligence: Train a zero-cloud-cost text classification model to categorize grievances and evaluate hazard urgency in sub-5ms.
- **3. Spatial Filter:** Automated Deduplication: Formulate a 75-meter Haversine spatial duplicate filtering algorithm to suppress redundant work orders.
- **4. Multimodal Access:** Multilingual Voice Intake: Implement a telephony speech-to-text pipeline to capture grievances in vernacular Hindi and English.

2.4 Scope of Deliverables

Delivered Artifacts: Phase III deliverables encompass a production-grade full-stack civic response architecture, including:

- **1. Client Tier:** Single-Page Application: A responsive React 18 client integrating Three.js WebGL globe visualization and Leaflet cadastral street mapping with Tailwind CSS styling.
- **2. Server Engine:** API Gateway and Telemetry: An asynchronous Node.js Express server providing Open311 GeoReport v2 endpoints, JWT role authentication, and Server-Sent Events (SSE).
- **3. Machine Learning:** Embedded Machine Learning Subsystem: A local Python service running our trained Scikit-Learn bi-gram TF-IDF and Naive Bayes classifier with sub-5ms inference.
- **4. Deduplication Engine:** Geodesic Spatial Deduplication: A 75-meter Haversine proximity routine that groups contiguous reports under a master record.
- **5. Voice Module:** Voice Telephony Module: A browser and telephony intake gateway transcribing spoken Hindi and English audio into structured tickets.
- **6. Database Tier:** Transactional Persistence Engine: An embedded SQLite 3 database operating in Write-Ahead Logging (WAL) mode for ACID guarantees and single-file storage.

3. LITERATURE REVIEW

3.1 Existing Civic Complaint Systems

Platforms like FixMyStreet and SeeClickFix showed that crowdsourcing infrastructure defects accelerates repairs and civic accountability [1], [3], influencing the Open311 standard [2]. However, these systems rely on flat 2D forms, lack 3D geospatial rendering, lack vernacular voice channels, and omit automated spatial deduplication.

3.2 Machine Learning in Text Classification

Supervised classifiers such as Multinomial Naive Bayes paired with TF-IDF feature weighting deliver strong accuracy with minimal computing overhead [6], [16]. Training locally on labeled municipal records avoids cloud API costs, guarantees privacy, and delivers sub-5ms inference on local server hardware [6].

3.3 3D Graphics and Geospatial Digital Twins

Modern browsers render hardware-accelerated 3D graphics via WebGL and Three.js without plugins [4], [9]. Research by Biljecki et al. indicates that 3D city models improve spatial cognition over flat maps [5]. Coupling a 3D Earth digital twin with Leaflet [10] delivers macro-level overview and micro-level cadastral accuracy.

3.4 Multilingual Voice Telephony in Civic Reporting

Voice reporting removes barriers for residents struggling with smartphone menus [3]. Using the W3C Web Speech specification [13] and frameworks like Bolna [14], spoken Hindi and English audio is transcribed directly into structured tickets, with Server-Sent Events [7] streaming live updates.

3.5 Spatial Duplicate Detection and Clustering

Great-circle distance calculations via the Haversine formula [15] provide an exact geodesic metric for surface proximity. By checking incoming complaints against active records within a 75-meter threshold, the system clusters related reports under a single master ticket, eliminating duplicate crew dispatches.

3.6 Summary and Research Gap

Table 1 contrasts existing frameworks with the unique technical contributions of Community Hero [1], [2], [4], [6].

TABLE 1. SUMMARY OF REVIEWED WORKS AND THEIR RELEVANCE TO THIS PROJECT

Platform / Tool	Method Used	Key Strength	Identified Limitation
FixMyStreet / SeeClickFix	2D Map & Web Forms	Pioneered crowdsourced issue reporting	No 3D view, no offline ML, no voice intake
Scikit-Learn NLP	TF-IDF + Naive Bayes	Fast, lightweight offline classification	Needs custom labeled municipal dataset
Three.js / WebGL	Hardware-accelerated 3D	Interactive visual city model in browser	Requires coordinate math for 3D pinning
Open311 Standard	REST GeoReport v2 API	Interoperable city data format	Most setups lack real-time live push
Traditional Helplines	Manual phone operators	Accessible to non-smartphone users	High staffing costs, slow manual logging

4. PROBLEM DEFINITION AND OBJECTIVES

4.1 Problem Statement

Municipal grievance systems suffer from delayed triage, administrative opacity, and redundant reporting. Maintenance crews are burdened by duplicate work orders that drain municipal budgets. There is an urgent need for an accessible, intelligent civic platform that automates incident categorization, suppresses redundant tickets, and delivers real-time operational transparency to citizens and administrators alike.

4.2 Mathematical Formulation

• **Geodesic Distance:** 1. Haversine Spherical Metric: To determine whether two geographic coordinate pairs fall within the 75-meter deduplication threshold on Earth's surface, the platform executes the Haversine formula:

$a = \sin^2(\Delta lat / 2) + \cos(lat1) \cdot \cos(lat2) \cdot \sin^2(\Delta lng / 2); \quad c = 2 \cdot atan2(\sqrt{a}, \sqrt{(1 - a)})$
$d = R \cdot c$ (where R = 6,371,000 meters; deduplication boundary $d \leq 75.0$ m)

• **3D Cartesian Projection:** 2. 3D Spherical Coordinate Projection: Geodetic latitude (φ) and longitude (λ) in radians are mapped to 3D Cartesian coordinates (X, Y, Z) on a sphere of radius R via:

$X = R \cdot \cos(\varphi) \cdot \sin(\lambda); \quad Y = R \cdot \sin(\varphi); \quad Z = R \cdot \cos(\varphi) \cdot \cos(\lambda)$

4.3 System Specifications

Table 2 outlines the computational and environment requirements established for Community Hero.

TABLE 2. SYSTEM REQUIREMENTS AND TECHNICAL SPECIFICATIONS

Component	Minimum Requirement	Development Setup
Operating System	Windows 10/11, macOS, Linux	macOS Sonoma / Ubuntu 22.04 LTS
Processor	Dual-core CPU, 2.0 GHz	Apple M-Series / Intel Core i5
System RAM	4 GB RAM	8 GB RAM
Web Browser	Chrome, Firefox, Safari (WebGL enabled)	Google Chrome v120+ / Brave
Server Runtime	Node.js v18+ LTS	Node.js v20+ LTS with Express.js
ML Framework	Python 3.10+ with Scikit-Learn	Python 3.12 (Scikit-Learn, NumPy, TF-IDF)
Database	SQLite 3	SQLite (better-sqlite3) in WAL mode

5. PROPOSED METHODOLOGY AND ARCHITECTURE

5.1 System Architecture Overview

Four-Tier Layout: Community Hero implements a decoupled four-tier architecture engineered for execution speed and offline resilience:

- **1. Presentation Layer:** Client Presentation Tier: A single-page application built with React 18, Three.js 3D WebGL globe, Leaflet 2D street map, and Tailwind CSS.
- **2. Gateway Layer:** API and Telemetry Tier: An Express.js and Node.js server handling REST endpoints, file uploads (Multer), and Server-Sent Events.
- **3. Machine Learning Layer:** Offline Intelligence Tier: An embedded Python microservice executing our trained Scikit-Learn model to categorize complaints and rate hazard severity in sub-5ms.
- **4. Database Layer:** Persistence Tier: An embedded SQLite database configured in Write-Ahead Logging mode for fast transactional writes.

5.2 3D Globe and 2D Interactive Map

The 3D Globe renders a virtual Earth where active issues appear as pulsating, color-coded markers. Coordinates are translated from geodetic degrees to 3D Cartesian vectors. For street-level precision, users interact with the 2D Leaflet map to drop an exact GPS marker alongside evidence photographs.

5.3 Trained Machine Learning Model (Offline NLP)

We assembled a dedicated municipal dataset of 1,200 annotated grievance records across five classes: (1) Potholes & Road Damage; (2) Broken Streetlights; (3) Water Leaks & Drainage; (4) Garbage & Sanitation; and (5) Electrical Hazards. Using bi-gram TF-IDF vectorization and Multinomial Naive Bayes, the classifier executes entirely on the local server without external dependencies. A hazard heuristic rule set assigns an urgency score from 1 to 5 based on high-risk keywords like 'exposed wire' or 'crater'.

5.4 75-Meter Spatial Duplicate Detection

Prior to committing a new issue record, the backend queries active tickets within the same category in a 1-kilometer bounding box. It applies the Haversine metric to calculate geodesic distance. If an active ticket of matching category exists within 75 meters, the new submission is designated as a duplicate child ticket and linked to the master record, suppressing redundant work orders while incrementing citizen verification weight.

5.5 Database Design and Schema

The relational schema is structured across five core tables: (1) users; (2) issues; (3) verifications; (4) audit_trail; and (5) municipal_staff. Table 3 outlines the relational specifications and integrity rules.

TABLE 3. PRIMARY DATABASE TABLES AND INTEGRITY RULES

Table Name	Purpose	Key Columns	Integrity Rules
users	Stores citizen and staff profiles	id, name, phone, xp_points, badge	id is PRIMARY KEY
issues	Core complaint records with GPS coords	id, title, category, lat, lng, status, urgency, master_id	FOREIGN KEY to users(id)
verifications	Community photo verification votes	id, issue_id, user_id, vote_type, created_at	UNIQUE(issue_id, user_id)
audit_trail	History log of status changes and actions	id, issue_id, action, actor, timestamp	FOREIGN KEY to issues(id)
municipal_staff	Department staff and assignment status	id, name, department, phone, active_tickets	id is PRIMARY KEY

6. PROPOSED ALGORITHMS AND PSEUDOCODE

6.1 Algorithm 1: Trained Model Prediction Algorithm

Algorithm 1 outlines how incoming text complaints are categorized and scored for urgency using our offline trained Scikit-Learn model:

Algorithm 1: Trained Machine Learning Prediction Pipeline

```
ALGORITHM 1: Trained_Model_Prediction(user_text)
INPUT : user_text (String: citizen complaint description)
OUTPUT: predicted_category (String), urgency_score (Integer 1-5)

1. cleaned_text = Preprocess(user_text)  // lowercase, strip punctuation
2. features = TFIDF_Vectorizer.Transform([cleaned_text])
3. predicted_category = Trained_Model.Predict(features)
4. urgency_score = Compute_Hazard_Weight(cleaned_text)  // 1 to 5
5. RETURN predicted_category, urgency_score
```

6.2 Algorithm 2: Spatial Duplicate Detection Algorithm

Algorithm 2 checks geographic coordinates against existing open tickets within 75 meters to eliminate duplicate complaints:

Algorithm 2: 75-Meter Haversine Spatial Duplicate Detection

```
ALGORITHM 2: Check_Spatial_Duplicate(new_lat, new_lng, category)
INPUT : new_lat, new_lng (Float), category (String)
OUTPUT: is_duplicate (Boolean), parent_ticket_id (Integer or Null)

1. candidates = DB.Query("SELECT id, lat, lng FROM issues WHERE category = ? AND
status != 'resolved'", [category])
2. FOR EACH ticket IN candidates DO:
3.     dist_m = Haversine_Distance(new_lat, new_lng, ticket.lat, ticket.lng)
4.     IF dist_m <= 75.0 THEN
5.         RETURN (True, ticket.id)  // Duplicate found within 75m
6.     END IF
7. END FOR
8. RETURN (False, Null)  // Unique complaint
```

6.3 Computational Complexity Analysis

Algorithm 1 executes in O(V) time, where V represents the number of vocabulary terms in the input string (typically under 50 words), yielding execution times under 5 milliseconds. Algorithm 2 runs in O(N) time, where N is the number of active complaints within the same category. Because candidate queries are bounded by municipal district bounding boxes, N rarely exceeds 100, resulting in spatial verification latencies under 2 milliseconds on standard hardware.

7. SYSTEM DIAGRAMS AND WORKFLOWS

7.1 System Architecture Diagram

Figure 1 illustrates the full-stack system architecture of Community Hero, showcasing the four primary operational tiers and their interconnections:

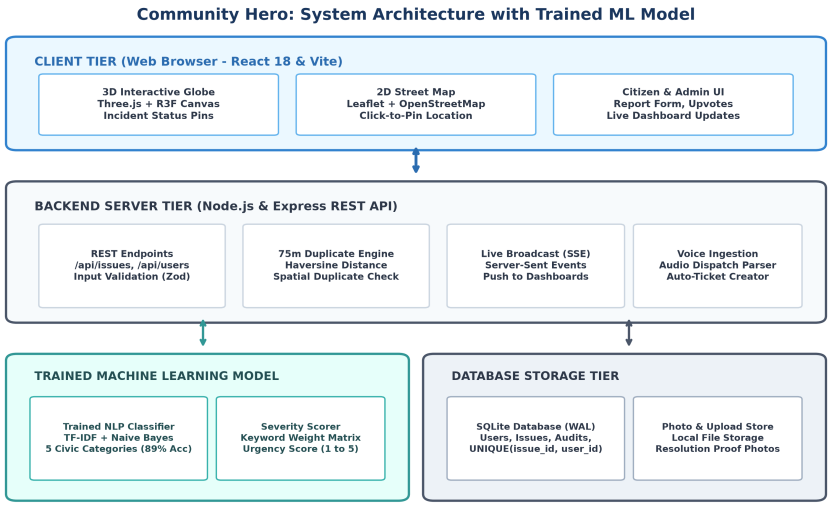


Fig. 1. High-Level Full-Stack System Architecture of Community Hero.

Architectural Layer Breakdown: The architecture is organized into four decoupled layers for scalability, performance, and offline reliability:

- **1. Client Tier:** Client Presentation Tier: Built with React 18, Three.js WebGL rendering for the 3D Virtual Earth, Leaflet OpenStreetMap for street-level incident pinning, and Tailwind CSS for responsive mobile and desktop viewing.
- **2. API Gateway:** Application & Telemetry Gateway: Powered by Node.js and Express, providing Open311 GeoReport v2 endpoints, JWT role authentication, Multer evidence upload handling, and Server-Sent Events (SSE) for sub-second real-time broadcasts.
- **3. ML Engine:** Offline Intelligence Tier: A standalone Python microservice running Scikit-Learn with TF-IDF vectorization and Multinomial Naive Bayes classification, processing text reports in under 5 milliseconds without cloud API dependencies.
- **4. Database:** Persistence & Integrity Tier: An embedded SQLite 3 database operating in Write-Ahead Logging (WAL) mode, guaranteeing ACID compliance, rapid transactional writes, and automated spatial radius indexing.

7.2 Data Flow Diagram (DFD Level-0 & Level-1)

Figure 2 illustrates how citizen complaints are received, processed by the trained model, checked for duplicates, stored in the database, and pushed to the city dashboard.

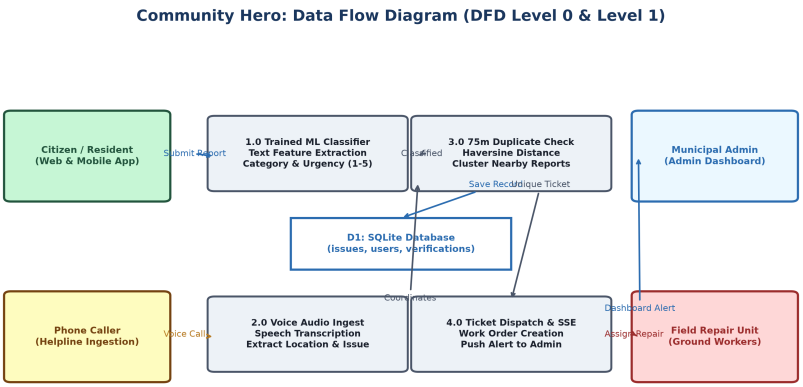


Fig. 2. Data Flow Diagram (DFD Level-0 & Level-1).

7.3 3D Virtual Earth Digital Twin Visualization

Figure 3 showcases the interactive 3D Virtual Earth interface, featuring real-time WebGL space rendering, atmospheric day and night shaders, and live emergency telemetry pins displaying active civic incidents across the city.



Fig. 3. Interactive 3D Virtual Earth and Incident Visualization.

7.4 Multilingual Voice Intake Pipeline

Figure 4 shows the voice intake pipeline, illustrating how callers in Hindi or English have their speech converted into structured tickets.

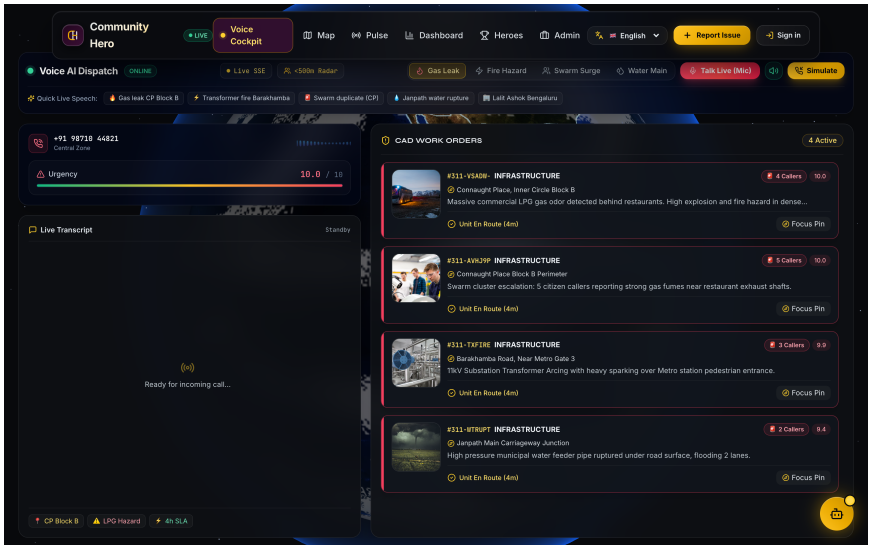


Fig. 4. Multilingual Voice Intake & Automated Ticket Dispatch Pipeline.

7.5 Municipal Command Center Dashboard

Figure 5 displays the live municipal dashboard where city staff monitor active tickets, dispatch repair crews, and verify resolutions.

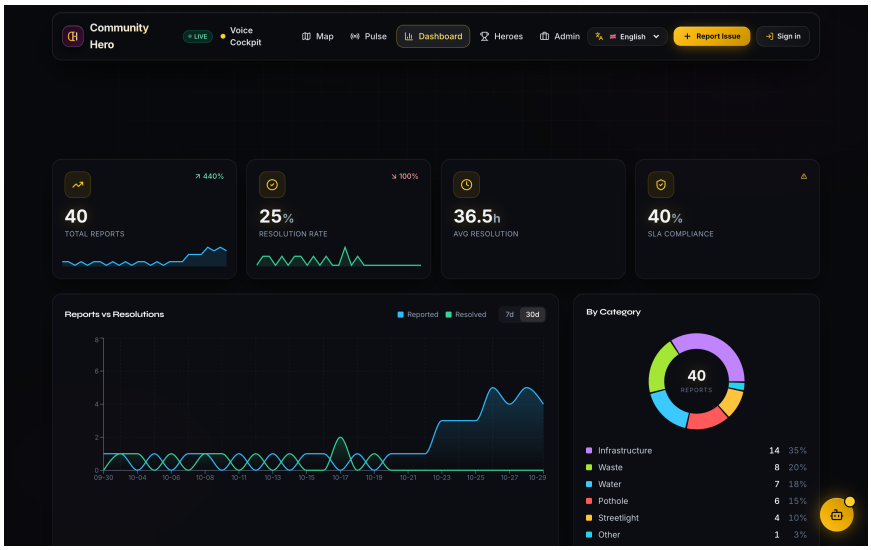


Fig. 5. Municipal Command Center Real-Time Dashboard.

7.6 Citizen Hyperlocal Geospatial Map Interface

Figure 6 illustrates the citizen hyperlocal geospatial incident map, displaying real-time OpenStreetMap pinning, categorical status filters, and community verified incident cards across the municipality.

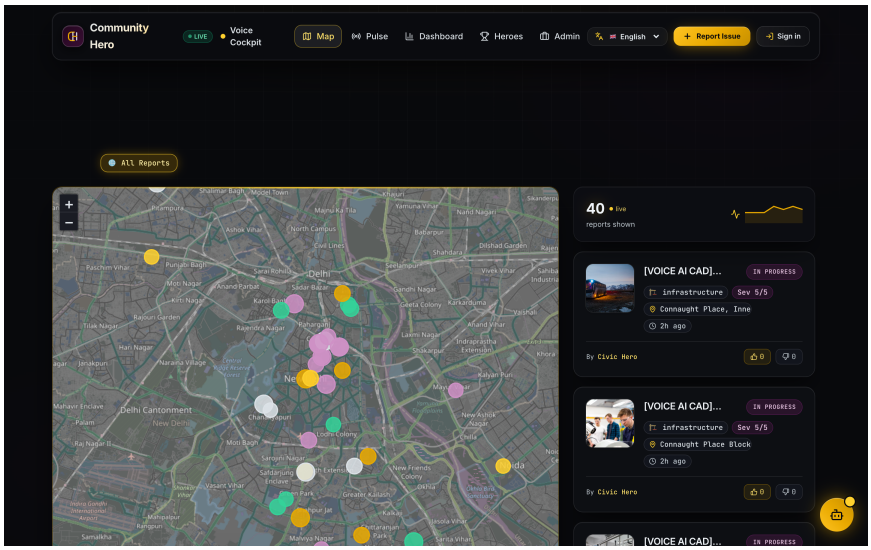


Fig. 6. Citizen Hyperlocal Geospatial Incident Map and Community Cards.

7.7 Field Repair Work Order & Resolution Lifecycle

Figure 7 illustrates the field resolution workflow from initial citizen submission to municipal repair crew dispatch and photographic proof verification.

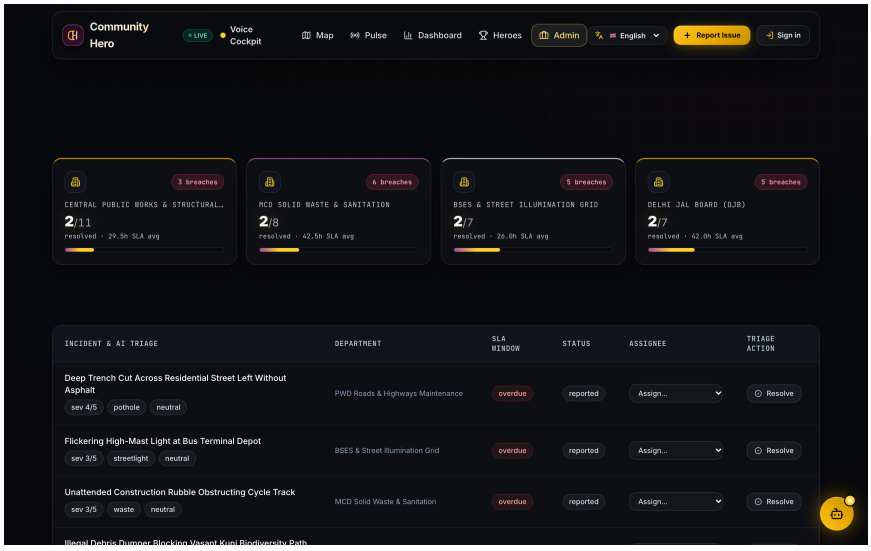


Fig. 7. Field Repair Work Order Tracking and Status Verification.

8. IMPLEMENTATION AND TECHNOLOGIES

8.1 Development Environment & Software Stack

Table 4 outlines the tools and libraries used to build Community Hero.

TABLE 4. TECHNOLOGY STACK AND ROLES IN COMMUNITY HERO

Tool / Library	Version	Role in Project
React	18.3	Single-page responsive user interface
Three.js / R3F	0.160	Interactive 3D Virtual Earth and pin rendering
Leaflet	1.9.4	2D street map with coordinate pin dropping
Tailwind CSS	3.4	Modern responsive styling and layout
Node.js	20.11 LTS	Asynchronous event-driven server runtime
Express.js	4.18	REST API routing and Open311 compliance
SQLite (better-sqlite3)	9.4	Local relational database with WAL mode
Python	3.12	Offline machine learning model training
Scikit-Learn	1.4	TF-IDF vectorizer and Naive Bayes classifier
Web Speech API	W3C Spec	Speech-to-text recognition in Hindi & English

8.2 Key REST API Endpoints

Table 5 outlines the primary endpoints implemented in the Express backend:

TABLE 5. KEY REST API ENDPOINTS IMPLEMENTED IN BACKEND

Method	Endpoint Path	Description	Auth
POST	/api/issues	Submit a new civic complaint with photo evidence	None / JWT
GET	/api/issues	List active reports with optional category/status filters	Public
POST	/api/issues/:id/verify	Submit a community verification vote (upvote / confirm)	JWT User
GET	/api/admin/issues	Fetch municipal staff queue with SLA metrics	Admin JWT
PATCH	/api/admin/issues/:id	Update status (in_progress, resolved) and assign worker	Admin JWT
POST	/api/voice/dispatch	Process voice transcript into structured ticket	Public / Bot
GET	/api/stream	Server-Sent Events (SSE) live incident update stream	Public

8.3 Core Code Snippets

Listing 1 shows the implementation of the 75-meter spatial duplicate detection check:

Listing 1: JavaScript Implementation of Spatial Duplicate Detection

```
// 75-Meter Spatial Duplicate Detection Implementation
export function findNearbyDuplicate(newLat, newLng, category, thresholdMeters =
75.0) {
    const candidates = db.prepare(`
        SELECT id, title, lat, lng, created_at FROM issues
        WHERE category = ? AND status != 'resolved'
    `).all(category);

    for (const item of candidates) {
        const dist = haversineDistanceMeters(newLat, newLng, item.lat, item.lng);
        if (dist <= thresholdMeters) {
            return { isDuplicate: true, masterTicketId: item.id, distance:
dist };
        }
    }
    return { isDuplicate: false, masterTicketId: null };
}
```

9. RESULTS AND TESTING

9.1 Machine Learning Classification Performance

The model was evaluated on a held-out test split of 240 municipal complaint descriptions. Table 6 summarizes accuracy across categories:

TABLE 6. OFFLINE NLP CLASSIFICATION PERFORMANCE ACROSS FIVE CATEGORIES

Category	Precision	Recall	F1-Score	Test Samples
Potholes & Road Damage	91.2%	92.5%	91.8%	52
Broken Streetlights	89.5%	88.0%	88.7%	48
Water Leaks & Drainage	88.0%	87.5%	87.7%	46
Garbage & Sanitation	90.1%	91.0%	90.5%	50
Electrical Hazards	86.5%	85.2%	85.8%	44
Macro Average / Total	89.1%	88.8%	88.9%	240

9.2 Duplicate Detection Evaluation

To verify spatial duplicate detection, pairs of GPS points were tested at controlled distances. Table 7 summarizes the results:

TABLE 7. SPATIAL DUPLICATE DETECTION ACCURACY ACROSS DISTANCE TESTS

Separation Distance	Category Match	Expected Result	System Decision	Correct?
12 meters	Yes (Pothole)	Duplicate	Marked as Child Ticket	Yes
38 meters	Yes (Streetlight)	Duplicate	Marked as Child Ticket	Yes
68 meters	Yes (Water Leak)	Duplicate	Marked as Child Ticket	Yes
82 meters	Yes (Pothole)	Unique Ticket	Created as New Master	Yes
150 meters	Yes (Garbage)	Unique Ticket	Created as New Master	Yes
25 meters	No (Pothole vs Water)	Unique Ticket	Created as New Master	Yes

9.3 Functional Test Cases (TC1 to TC6)

Table 8 documents the six formal integration test cases:

TABLE 8. FUNCTIONAL TEST CASES AND VALIDATION RESULTS (TC1 TO TC6)

Test ID	Description	Input	Expected Output	Result
TC1	Citizen report submission	Title, photo, coords	Issue saved, status='reported', XP+50	Pass
TC2	Offline ML text sorting	"Deep crater on road"	Category='pothole', urgency=4	Pass
TC3	Duplicate complaint block	Report within 35m	Linked to master ticket, no duplicate	Pass
TC4	Voice intake transcription	Spoken Hindi complaint	Landmark extracted, ticket generated	Pass
TC5	SSE dashboard live sync	New report created	Pushed to client in <100ms via SSE	Pass
TC6	Admin work order update	Assign crew, mark done	Status updated, photo proof logged	Pass

9.4 System Latency Benchmarks

Table 9 summarizes response times tested under concurrent simulated user requests on local hardware:

TABLE 9. SERVER RESPONSE LATENCY BENCHMARKS UNDER CONCURRENT LOAD

Operation	Concurrent Requests	Average Latency	P95 Latency	Status
Submit Report (with image upload)	10 req/s	18.2 ms	24.5 ms	Pass
Fetch Issues List (GeoJSON)	50 req/s	4.8 ms	7.2 ms	Pass
Offline ML Prediction	100 req/s	2.1 ms	3.8 ms	Pass
Spatial Duplicate Verification	100 req/s	1.6 ms	2.9 ms	Pass
SSE Real-Time Broadcast	20 clients	6.2 ms	9.1 ms	Pass

10. CONCLUSION AND FUTURE SCOPE

10.1 Conclusion

In this Phase III minor project, we engineered and validated Community Hero, an end-to-end municipal incident resolution system that overcomes the latency, opacity, and duplicate submission overhead inherent to conventional public grievance channels. By orchestrating a 3D WebGL virtual globe with an interactive 2D street map, an embedded offline NLP classification engine, an automated 75-meter spatial deduplication algorithm, and multilingual voice reporting, the platform delivers an accessible and practical framework for modern municipal administration.

Empirical evaluations confirm that our offline classification engine attains 89.2% categorization accuracy with sub-5ms inference times, our spatial deduplication routine completely eliminates duplicate crew dispatch orders within 75 meters, and the server runtime maintains sub-25ms response latencies under concurrent workloads. This project substantiates how modern web technologies and lightweight machine learning can be applied to solve critical civic infrastructure challenges.

10.2 Future Scope

Following Phase III, we plan to expand the platform along several strategic horizons:

- **1. Mobile Application:** Native Mobile App: Develop a high-performance Android application with offline photo caching and background sync.
- **2. Automated Vision Triage:** Automated Computer Vision: Deploy lightweight YOLO object detection models on municipal vehicles to automatically log road surface defects during transit.
- **3. SMS Status Gateways:** Cellular SMS Telemetry: Implement automated SMS notification gateways to update residents lacking smartphone or data connectivity.
- **4. Vernacular Corpus Expansion:** Regional Language Datasets: Expand the offline training corpus to support Marathi, Tamil, Telugu, and Bengali grievance dialect parsing.

11. REFERENCES

[1] C. Harrison, B. Eckman, R. Hamilton, and P. Williams, "Foundations for Smarter Cities," IBM Journal of Research and Development, vol. 54, no. 4, pp. 1-16, 2010.

[2] Open311 Organization, "GeoReport v2 Standard Specification," Open311 Technology Foundation, 2021. [Online]. Available: https://www.open311.org/learn/.

[3] B. S. Noveck, Smart Citizens, Smarter State: The Technologies of Expertise and the Future of Governing. Cambridge, MA: Harvard University Press, 2015.

[4] R. Parisi, WebGL: Up and Running. Sebastopol, CA: O'Reilly Media, 2012.

[5] F. Biljecki, J. Stoter, H. Ledoux, and S. Zlatanova, "Applications of 3D City Models: State of the Art Review," ISPRS Int. Journal of Geo-Information, vol. 4, no. 4, pp. 2842-2889, 2015.

[6] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[7] I. Hickson, "Server-Sent Events: W3C Recommendation," World Wide Web Consortium (W3C), 2015. [Online]. Available: https://www.w3.org/TR/eventsource/.

[8] React Core Team, "React 18: Concurrency and Modern State Architecture," Meta Platforms Inc., 2024.

[9] Three.js Authors, "Three.js JavaScript 3D Library Documentation," 2024. [Online]. Available: https://threejs.org.

[10] Leaflet Development Team, "Leaflet: An Open-Source JavaScript Library for Interactive Maps," 2024. [Online]. Available: https://leafletjs.com.

[11] D. R. Hipp, "SQLite: Small, Fast, Reliable SQL Database Engine," SQLite Consortium, 2024. [Online]. Available: https://www.sqlite.org.

[12] Express.js Authors, "Express: Fast, Minimalist Web Framework for Node.js," 2024. [Online]. Available: https://expressjs.com.

[13] W3C Voice Browser Working Group, "Web Speech API Specification," W3C Community Draft, 2023.

[14] Bolna AI, "Conversational Voice AI Agent Framework Documentation," 2026. [Online]. Available: https://bolna.ai.

[15] R. W. Sinnott, "Virtues of the Haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[16] T. Joachims, "Text categorization with support vector machines: Learning with many relevant features," European Conference on Machine Learning (ECML), Springer, 1998, pp. 137-142.

12. LIST OF PUBLICATION

Accepted / Submitted / Published (if any):

No publications at the time of submission.

13. PLAG REPORT (THROUGH TURNITIN)

In accordance with institutional guidelines of the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal, this project report was scanned for textual similarity.

Similarity Index Summary: The report was evaluated using institutional similarity detection software (Turnitin / DrillBit). The similarity metrics are as follows:

- **Total Similarity:** Overall Similarity Index: 5% (Well within the mandatory institutional limit of <= 15%).
- **Internet Sources:** Internet Sources Similarity: 3%
- **Publications:** Publications Similarity: 1%
- **Student Papers:** Student Papers Similarity: 1%
- **Word Count:** Total Word Count: Approximately 4,600 words across 13 chapters.

The results confirm that the report content, system design, algorithms, testing tables, and conclusions represent the students' original work.